Documentation Sentinel App

Furkan Ciraci

Stagiaire I.S. Informatique

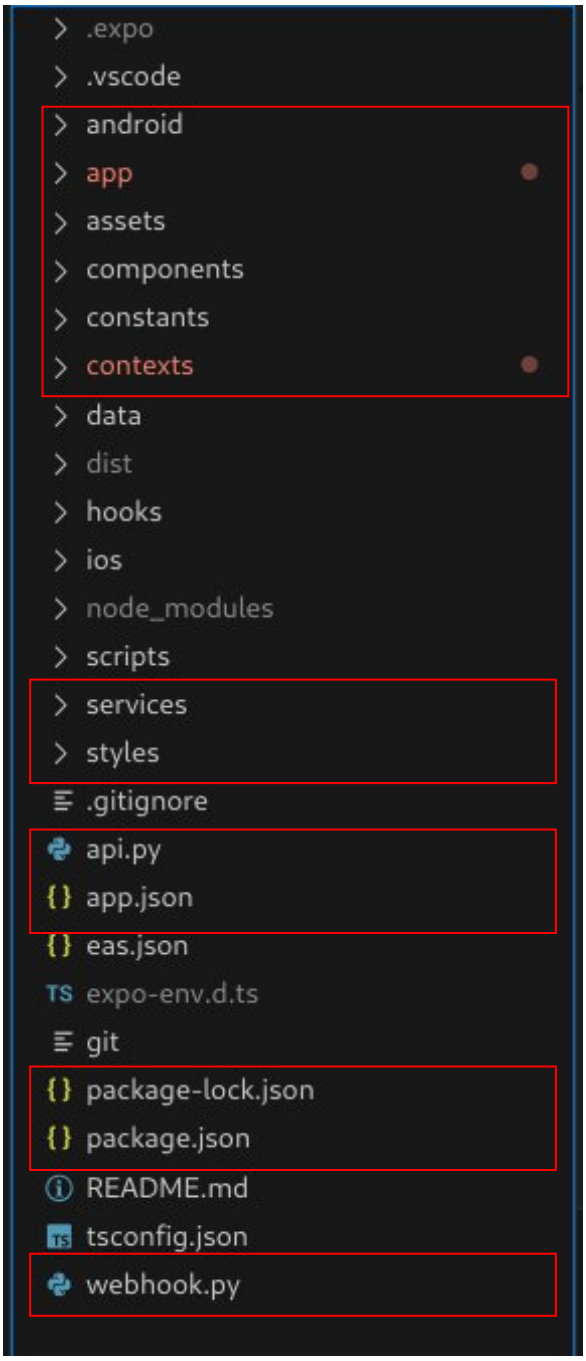
2025

Furkan Ciraci

2025

1. Structure de l'application 3
1.1 Le dossier Android 4
1.1.1 Le fichier build.gradle 5
1.1.2 Générer les .apk et .aab 6
1.1.3 Générer le site web 7
1.1.4 Modifier les logos de l'application 8
1.2 Le fichier app.json et package.json 10
1.3 L'API et le Webhook 11
1.4 Les Services 11
1.5 Le context 12
1.6 Les components 13
1.7 Le dossier apps 14
1.8 Les constants 14
1.9 Les styles 15
1.10 Les assets 15
2. Pour les tests et debug 16
2.1 Sur le web 16
2.2 Mise à jour OTA (Over-The-Air) 16

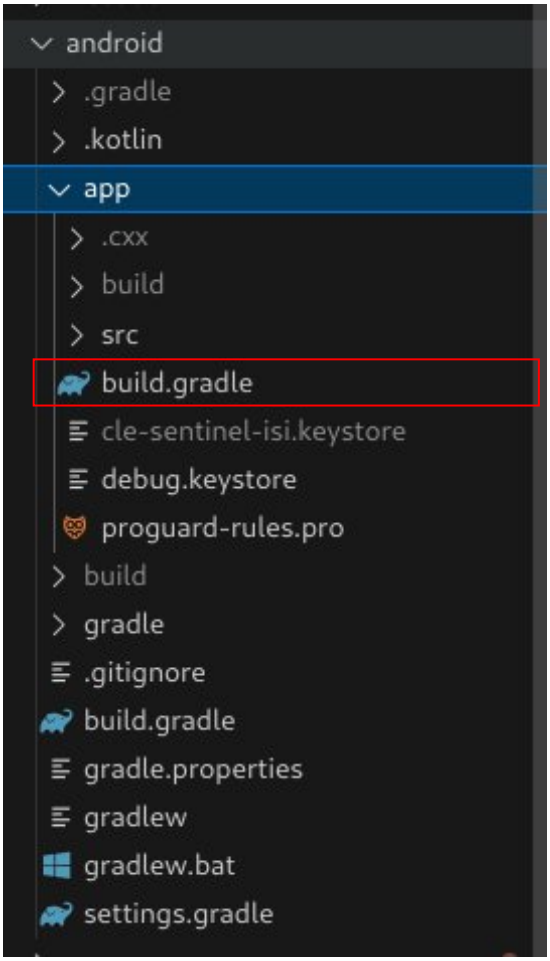
1. Structure de l’application



Sont entouré en Rouge les fichiers importants de l’application. Je vais les détailler ci-dessous.

⚠ BIEN FAIRE NPM INSTALL À LA PREMIÈRE OUVERTURE DU DOSSIER !

1.1 Le dossier Android



Composant essentiel de l'application, c'est de lui que va sortir la version finale, celle qu'on aura sur nos téléphones (android).
Le dossier le plus important est le dossier app/ c'est dans celui ci qu'on trouvera nos build que ça soit APK ou .aab et c'est ici qu'il faudra modifier le numéro de version dans le fichier build.gradle pour chaque MAJ.

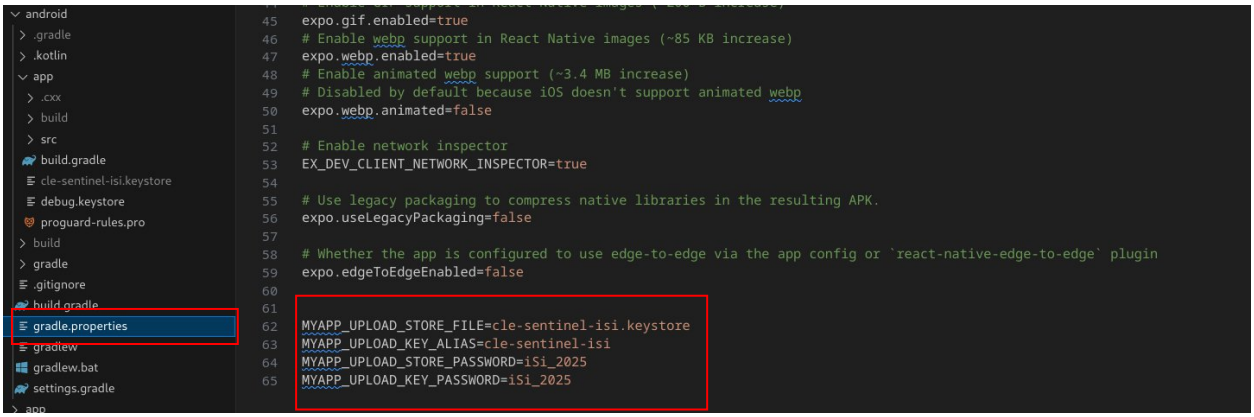
1.1.1 Le fichier build.gradle

```
build.gradle
android > app > build.gradle > android
79   * give correct results when using with locales other than en-US. Note that
80   * this variant is about 6MiB larger per architecture than default.
81   */
82  def jscFlavor = 'io.github.react-native-community:jsc-android:2026004.+'
83
84  android {
85      ndkVersion rootProject.ext.ndkVersion
86
87      buildToolsVersion rootProject.ext.buildToolsVersion
88      compileSdk rootProject.ext.compileSdkVersion
89
90      namespace 'com.boudier.isisentinel'
91      defaultConfig {
92          applicationId 'com.boudier.isisentinel'
93          minSdkVersion rootProject.ext.minSdkVersion
94          targetSdkVersion rootProject.ext.targetSdkVersion      You, 2 weeks ago • AJout fichier manquant IOS et ANDROID …
95          versionCode 6
96          versionName "1.0.6"
97      }
98      signingConfigs {
99          release {
100             storeFile file(MYAPP_UPLOAD_STORE_FILE)
101             storePassword MYAPP_UPLOAD_STORE_PASSWORD
102             keyAlias MYAPP_UPLOAD_KEY_ALIAS
103             keyPassword MYAPP_UPLOAD_KEY_PASSWORD
104         }
105     }
106     buildTypes {
107         debug {
```

```
app.json
app.json > {} expo > {} web
You, 16 hours ago | 1 author (You)
1   {
2     "expo": {
3       "name": "iSi-Sentinel",
4       "slug": "iSi-Sentinel",
5       "version": "1.0.6",
6       "orientation": "portrait",
7       "icon": "./assets/images/iSi_sentinel_logo.jpg",
8       "scheme": "myapp",
9       "userInterfaceStyle": "automatic",
10      "newArchEnabled": true,
11      "ios": {
12        "supportsTablet": true,
13        "bundleIdentifier": "com.boudier.isisentinel"
14      },
15      "android": {
16        "package": "com.boudier.isisentinel",
17        "versionCode": 6,
18        "adaptiveIcon": {
19          "foregroundImage": "./assets/images/iSi_sentinel_logo.jpg"
20        }
21      },
22      "web": {
23        "bundler": "metro",
24        "output": "static",      You, last month • commit initial …
25        "favicon": "./assets/images/favicon.png"
```

Il faut ici faire attention à sélectionner le bon fichier build.gradle car il y en a un autre auquel on ne fera pas attention, le bon se trouve à ce chemin : ./android/app/build.gradle
Entouré en Rouge se trouve le numéro de version qu'il faudra soigneusement modifié à chaque mise à jour, ce numéro de version devra également être modifié dans le fichier app.json qui se trouve à la racine du projet.

En dessous, entouré en violet, se trouvent les signingConfigs qui servent à configurer la signature de l'application Android pour la version release, celle qui sera sur le PlayStore. Les variables (mots de passes etc. se trouvent tout en bas du fichier gradle.properties qui lui va se trouver à la racine du fichier android : android/gradle.properties



1.1.2 Générer les .apk et .aab

Pour générer les .apk et .aab il faut **impérativement,** dans un terminal, se placer dans le dossier /android pour avoir accès a l'éxecutable ./gradlew.
⚠ Bien faire ./gradlew clean avant tout build pour être sûr de supprimer l'ancienne !
Une fois qu'on est dans le dossier /android on peut librement executer nos 2 commandes :

- ./gradlew assembleRelease : Nous permet d'obtenir un .apk qui se trouve dans le dossier ./android/app/build/outputs/apk/release/app-release.apk
- ./gradlew bundleRelease : Nous permet d'obtenir un .aab qui se trouve dans le dossier ./android/app/build/outputs/bundle/release/app-release.aab

Rappel : Les .apk sont uniquement réservé pour les tests, pour publier sur le Play Store il faut passer par un .aab

Optionnel : On peut également build des apk et des aab avec expo, celà évite notamment d'utiliser les ressources du PC en cas de petites config. Pour cela il faut d'abord installer eas-cli : npm install -g eas-cli
Puis il faut se connecter à son compte expo : eas login
Après la connexion il faut initialiser EAS : eas build:configure
Une fois toutes ses étapes accomplis on peut build nos .apk et .aab :

- eas build --platform android --profile production : pour générer un .aab prêt à la publication sur le Play Store
- eas build --platform android --profile preview : pour générer un .apk

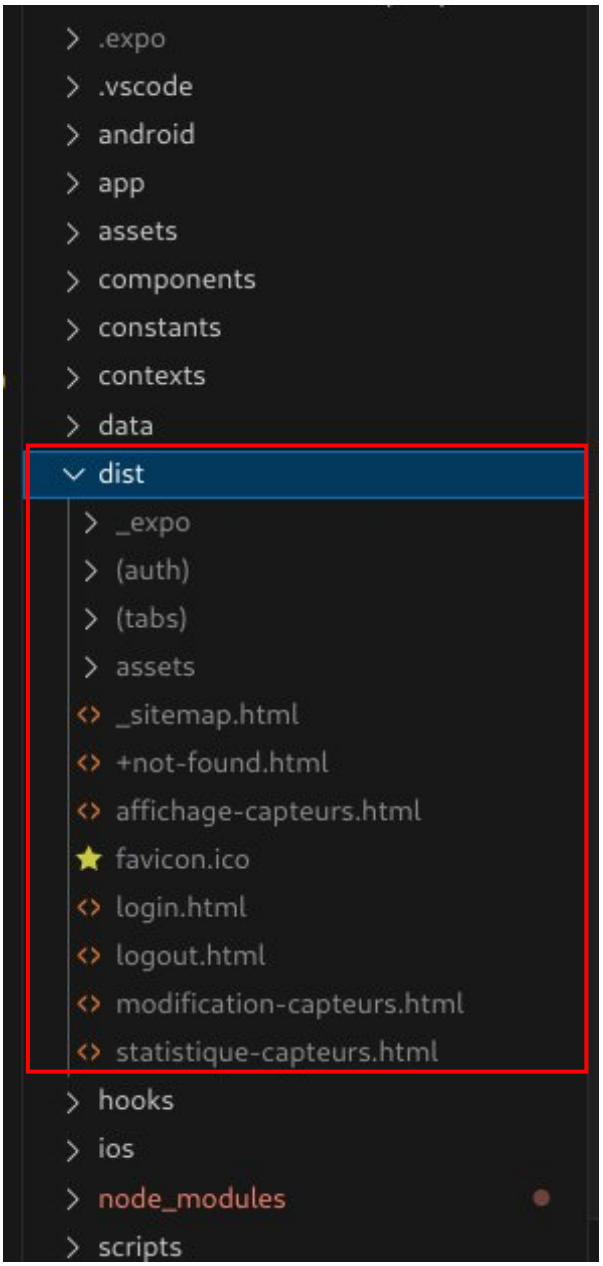
⚠ Attention : on ne peut pas faire un nombre de build illimité, avec la version gratuite on à 50 build, de plus on risque de rencontrer une file d'attente de plusieurs heures, il est donc préférable de build avec ./gradlew assembleRelease et ./gradlew bundleRelease.

1.1.3 Générer le site web

Pour générer le site web il suffit de se placer à la racine du projet (pour bénéficier des outils d'expo) et exécuter la commande :

```
npx expo export --platform web
```

Tout les fichiers seront générer dans le dossier dist.



Pour pouvoir l’exporter sur web faire ces étapes :

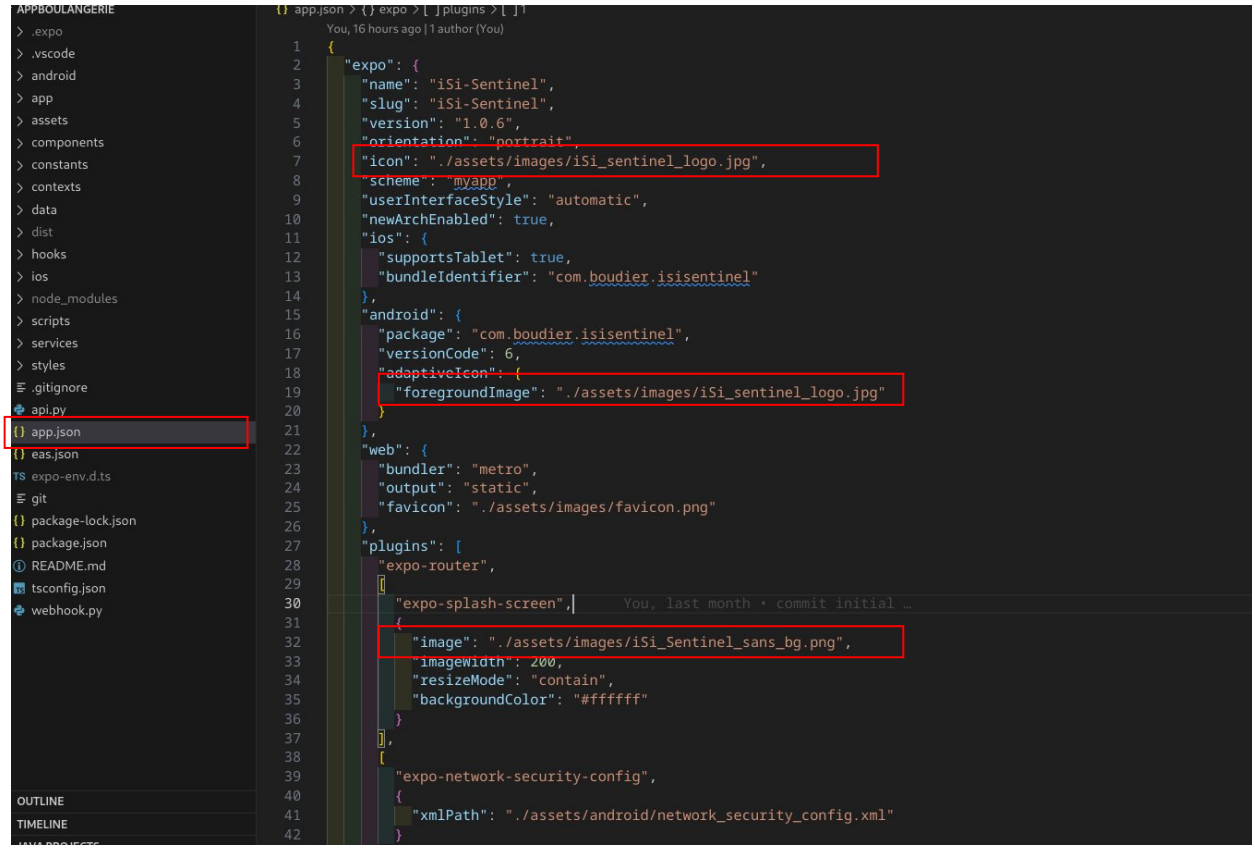
1. sudo mkdir -p /var/www/iot-sentinel : crée le dossier d’accueil qui contiendra le contenu du dist.
2. sudo chown -R www-data:www-data /var/www/iot-sentinel : Pour que Nginx puisse lire les fichiers, on donne les droits à www-data
3. sudo nano /etc/nginx/sites-available/iot-sentinel : crée le fichier de configuration NGINX, sur le serveur il y sera déjà il suffit de le modifier au besoin.
4. sudo ln -s /etc/nginx/sites-available/iot-sentinel /etc/nginx/sites-enabled/ : Activer le site ⚠ commande en une ligne faire attention
5. sudo nginx -t : tester la configuration
6. sudo systemctl reload nginx : recharger le serveur

Optionnel : sudo tail -f /var/log/nginx/iot-sentinel-error.log : consulter les logs en cas de problèmes.

1.1.4 Modifier les logos de l’application

Pour modifier Les logos et autres images de l’application, il faut tout d’abord les placer dans le fichier ./assets/images.
Il faut ensuite les changer dans le app.json : voir ligne entourer en rouge.
Icon est l’icône qui s’affiche sur le menu du téléphone
Expo-splash-screen “image” est l’image qui s’affiche au démarrage de l’application
Etc.
Une fois ces changements fait il faut régénérer le dossier android pour que les changements soient effectifs avec la commande : npx expo prebuild --platform android

Furkan Ciraci



2025

1.2 Le fichier app.json et package.json

Trouvable à la racine du projet, le fichier app.json gère tout ce qui est caractéristique de l'application comme le nom, les logo, les versions etc.
Le fichier package.json gère lui les différents paquets installés avec leur version, important à vérifier en cas de conflits notamment lorsqu'un paquet va être obsolète ou qu'une mise à jour a été faite.

```
{} app.json ×
{} app.json > {} expo > [ ] plugins > [ ] 1
You, 17 hours ago | 1 author (You)
1  {
2    "expo": {
3      "name": "iSi-Sentinel",
4      "slug": "iSi-Sentinel",
5      "version": "1.0.6",
6      "orientation": "portrait",
7      "icon": "./assets/images/iSi_sentinel_logo.jpg",
8      "scheme": "myapp",
9      "userInterfaceStyle": "automatic",
10     "newArchEnabled": true,
11     "ios": {
12       "supportsTablet": true,
13       "bundleIdentifier": "com.boudier.isisentinel"
14     },
15     "android": {
16       "package": "com.boudier.isisentinel",
17       "versionCode": 6,
18       "adaptiveIcon": {
19         "foregroundImage": "./assets/images/iSi_sentinel_logo.jpg"
20       }
21     },
22     "web": {
23       "bundler": "metro",
24       "output": "static",
25       "favicon": "./assets/images/favicon.png"
26     },
27     "plugins": [
28       "expo-router",
29       [
30         "expo-splash-screen",      You, last month • commit initial …
31         {
32           "image": "./assets/images/iSi_Sentinel_sans_bg.png",
33           "imageWidth": 200,
34           "resizeMode": "contain",
35           "backgroundColor": "#ffffff"
36         }
37       ],
38       [
39         "expo-network-security-config",
40         {
41           "xmlPath": "./assets/android/network_security_config.xml"
42         }
43       ]
```

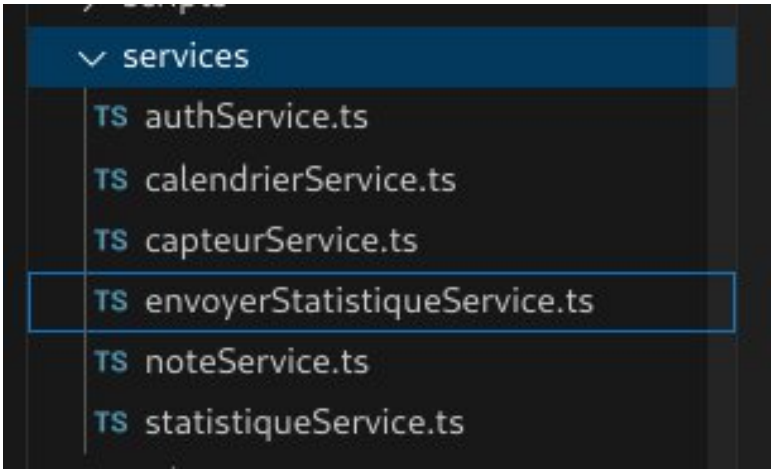
```
{} package.json 1 ×
{} package.json > {} dependencies > expo-splash-screen
You, 6 days ago | 1 author (You)
1  {
2    "name": "iSi-Sentinel",
3    "main": "expo-router/entry",
4    "version": "1.0.0",
     ▷Debug
5    "scripts": {
6      "start": "expo start",
7      "reset-project": "node ./scripts/reset-project.js",
8      "android": "expo run:android",
9      "ios": "expo run:ios",
10     "web": "expo start --web",
11     "test": "jest --watchAll",
12     "lint": "expo lint"
13   },
14   "jest": {
15     "preset": "jest-expo"
16   },
17   "dependencies": {
18     "@expo/vector-icons": "^14.1.0",
19     "@react-native-async-storage/async-storage": "2.1.2",
20     "@react-native-community/datetimepicker": "^8.3.0",
21     "@react-native-picker/picker": "^2.11.0",
22     "@react-navigation/bottom-tabs": "^7.2.0",
23     "@react-navigation/native": "^7.0.14",
24     "axios": "^1.8.4",
25     "date-fns": "^4.1.0",
26     "eas-cli": "^16.4.1",
27     "expo": "^53.0.8",
28     "expo-blur": "~14.1.4",
29     "expo-constants": "~17.1.5",
30     "expo-font": "~13.3.1",
31     "expo-haptics": "~14.1.4",
32     "expo-linking": "~7.1.4",
33     "expo-network-security-config": "^1.0.2",
34     "expo-router": "5.0.6",
35     "expo-splash-screen": "~0.30.8",      You, 3 weeks ago • point de sauvegarde …
36     "expo-status-bar": "~2.2.3",
37     "expo-symbols": "~0.4.4",
38     "expo-system-ui": "~5.0.7",
39     "expo-updates": "~0.28.13",
40     "expo-web-browser": "~14.1.6",
41     "qs": "^6.14.0",
42     "react": "19.0.0",
```

1.3 L’API et le Webhook

L’API va gérer la connexion au serveur et à la base de données pour l’application, il va récupérer les capteurs, les notes etc.
Le Webhook gère la récupération des données depuis la passerelle, il s’occupe également de l’envoie des mails d’alertes et de stats.

1.4 Les Services

Les services sont un point essentiels de l’application, ce sont eux qui vont faire le lien entre le front et le back, il vont directement communiquer avec le serveur.
Exemple : j’appuie sur le bouton envoyer mail pour les stats, ce bouton va actionner une fonction dans le service envoyerStatistiqueService.ts qui va communiquer avec l’API pour envoyer le mail.

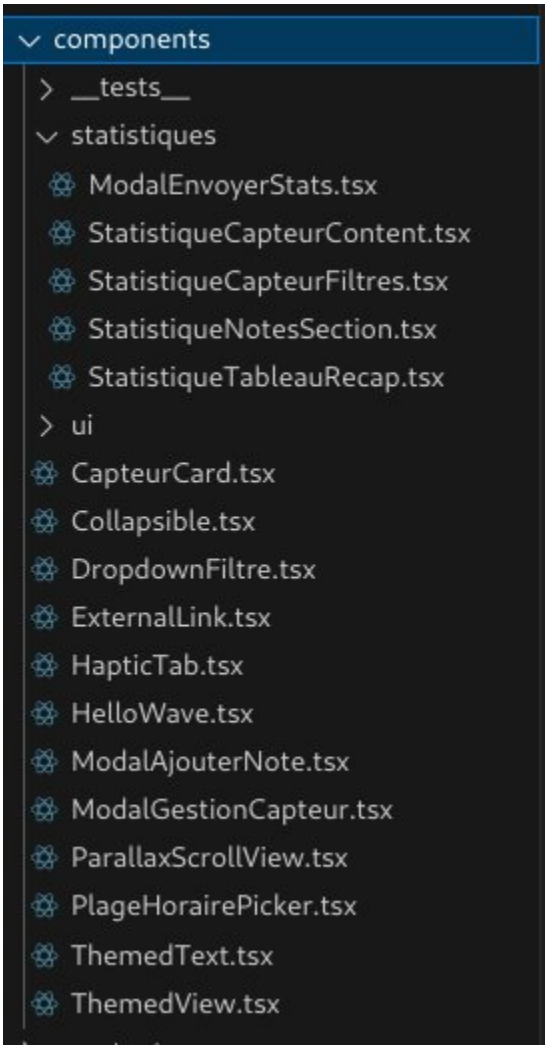


1.5 Le context

Le context dans React permet de partager facilement des informations entre plusieurs parties de l’application sans avoir à tout transmettre manuellement. Dans notre cas, il sert à centraliser toutes les données liées aux capteurs (température, humidité, batterie…), ainsi que les filtres, les dates sélectionnées, les sites ou capteurs choisis. Grâce à lui, tous les composants peuvent accéder à ces informations et les mettre à jour de manière simple et cohérente, ce qui rend l’application plus claire, plus fluide et plus facile à maintenir.

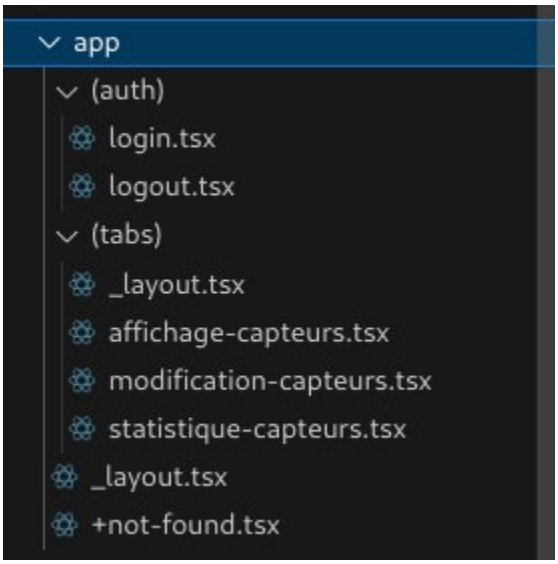
1.6 Les components

Ce sont les éléments visuels de l'application, ce sont pas exemple les filtres, les modales, les capteurs, ils permettent de décharger les fichiers des pages et permettent de réutiliser les mêmes éléments visuels dans plusieurs fichiers sans avoir à les re développer 2, 3, 4 fois.



Exemple CapteurCard.tsx est utilsé pour afficher, les capteurs sous formes de carte carrée, il est utilisé dans le fichier d'affichage et de modifications des capteurs.

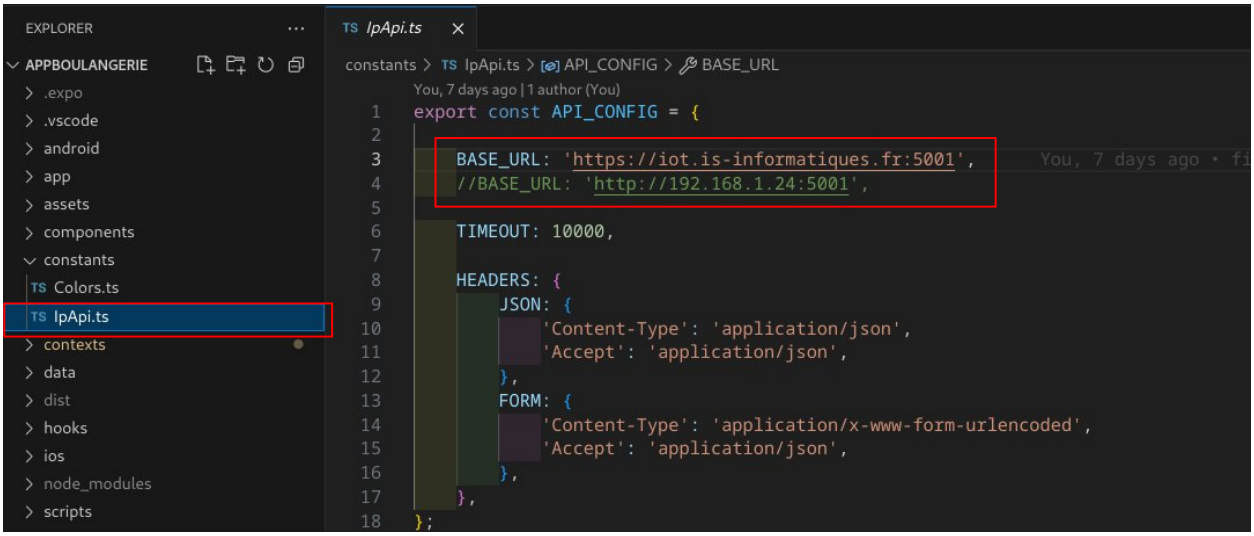
1.7 Le dossier apps



Il contient toutes les pages de l'application, celle dans lesquelles on peut naviguer. Les pages sont constituées de différents components mis ensemble. C'est le front-end de l'application.

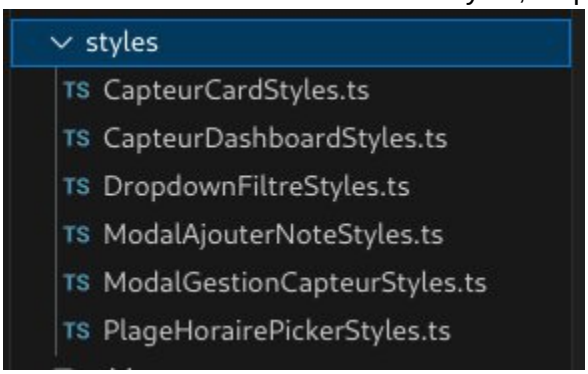
1.8 Les constants

Le fichier le plus importants à l'intérieur est IpApi.ts il permet de modifier l'ip de l'API dans un seul fichier, une seule ligne, pour que le changement soit effectif partout.



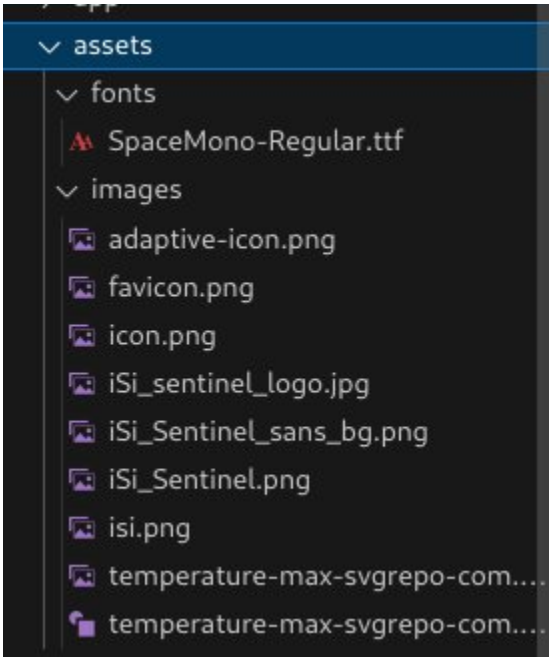
1.9 Les styles

Ce sont les différentes feuilles de styles, l’équivalent des CSS mais en TypeScript.



1.10 Les assets

Ce sont tous simplement les différentes images et polices d’écriture :



2. Pour les tests et debug

2.1 Sur le web

Pour faire des tests de l’application sur web exécuter la commande npx expo start.

À distance il faut crée un tunnel avec l’option : --tunnel, npx expo start --tunnel.

2.2 Mise à jour OTA (Over-The-Air)

Avec expo on peut push des mises à jour de test expo publish, cette commande permet également de push une mise à jour si on a fait un build avec expo, ce qui n’est pas notre cas.